
Structured Rule Variation as Pressure for Modularization in Transformers

E. Spencer Shelton
University of Chicago
spencershelton@uchicago.edu

Abstract

Transformers often represent behavior in densely shared parameter spaces, making it difficult to target specific aspects of a model after training. This paper studies whether structured variation over related rule systems can create training-time pressure for modularization in transformers. Chess provides a controlled setting for studying this; its rules are explicit and programmatically modifiable, and synthetic data can be generated with precisely defined, targeted rule changes. We train transformer models on next-move prediction across both standard chess games and games drawn from a structured family of per-piece movement-rule variants, with the models including piece-specific module slots at fixed positions in the architecture associated with piece-rule variants. The central hypothesis is that exposure to many related per-piece rule systems, paired with architectural constraints that limit shared computation and select rule-specific modular capacity according to the current ruleset, can encourage rule-specific behavior to localize in modular components. We evaluate this hypothesis using rule-conditional performance, targeted ablations, module swaps, and comparisons among modular architectures. As a stronger downstream test, we examine whether held-out movement rules can be incorporated by training only newly introduced module slots, and measure both gains on rule-specific moves and interference on unchanged parts of play. More broadly, this project tests whether rule-family training can induce functional modularity in transformers, with possible implications for lower-interference adaptation in highly structured, rule-governed domains such as programming languages.

1 Introduction

Transformers can be trained as dense networks where examples from many contexts update a shared parameter space, and downstream behaviors emerge from interactions among many layers and attention heads. This creates a practical difficulty that after training as it can be unclear how one might target a specific behavior without affecting many unrelated behaviors. This paper asks whether the training distribution itself can be structured to pressure a transformer toward reusable modular components.

This question is studied in the context of chess. Chess is useful here not because chess play is the final application, but as it gives a controlled rule-governed environment where rules are explicit, legal move generation can be checked programmatically, and synthetic games can be generated under precisely specified rule changes. For this paper a family of chess-like games was constructed in which individual piece movement rules are changed and the model is trained for next-move prediction over standard chess and over games sampled from combinations of per-piece rule variants using a specialized vocabulary designed to represent the games. The model is not given any indicator in the input of what ruleset is active, instead the training scaffold selects module slots associated with the active per-piece rule variants.

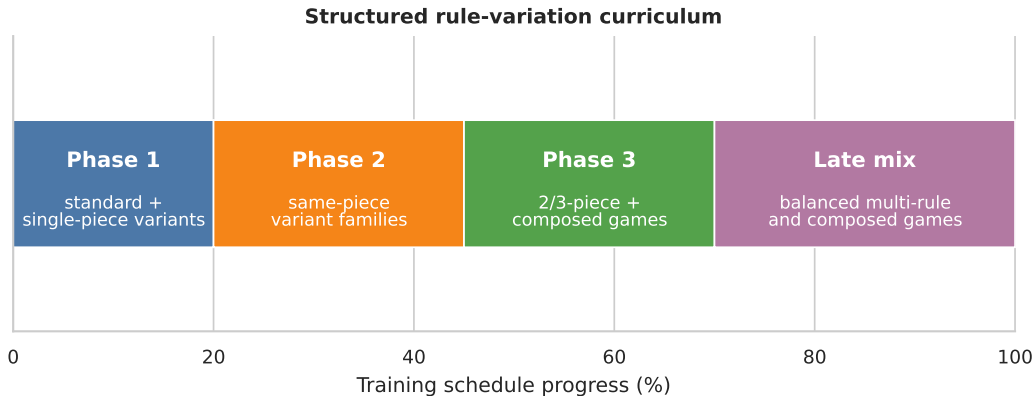


Figure 1: Training curriculum. The data schedule moves from standard and single-piece variants toward multi-rule and composed games, increasing pressure to represent rule-dependent behavior.

The motivating hypothesis is that structured rule variation can make it useful for the network to store rule-specific behavior in corresponding module slots. If this occurs, then changing or adding a rule should be possible by changing or training a small part of the model, with limited interference elsewhere. This paper asks whether module slots carry functionally relevant information, whether wrong module selections damage the relevant behavior, and whether held-out rules can be learned by training newly introduced module slots.

The contributions this paper attempts to make are:

- Introducing a controlled rule-family training setup based on per-piece chess movement variants.
- Comparing three modular transformer variants: a two-stage sequence model, a single-stage sequence model, and a position-only single-stage model.
- Evaluating modularity using held-out rule adaptation, wrong-module stress tests, rule-conditional performance, and regression on unchanged parts of play.
- Finding evidence for partial functional localization: held-out modules are necessary for unique held-out move geometry, but unpolished modules can interfere with unchanged context moves. A short polish pass reduces this interference substantially.

2 Setup: structured chess rule variation

Let a ruleset be represented by a vector of six piece-rule variant identifiers, one for each piece type. The standard ruleset has all identifiers set to zero and a nonzero entry changes the movement rule for one piece type while leaving the rest of the game structure intact. Training games include standard chess, single-piece variants, multi-piece variants, and composed games that splice games at shared positions with composition exposing the model to contexts in which the same board position can appear under different rule histories and rule combinations.

Two movement rules are held out from the main rule-family training and used for adaptation tests: a king rule allowing a two-square diagonal move (K5), and a knight rule allowing a two-by-four leap (N5). These rules create move geometries absent from the seen rule families and allows us to define what should be a reasonably strong diagnostic: on positions where the target move is legal only under the held-out rule geometry, performance without the corresponding held-out module should be poor, while performance with a trained held-out module should improve.

3 Models

All models are decoder-style transformers trained for next-token next-move prediction. Moves are tokenized into structured components: piece, source square, target square, and termination or promotion tokens. We study three model families (Figure 2).

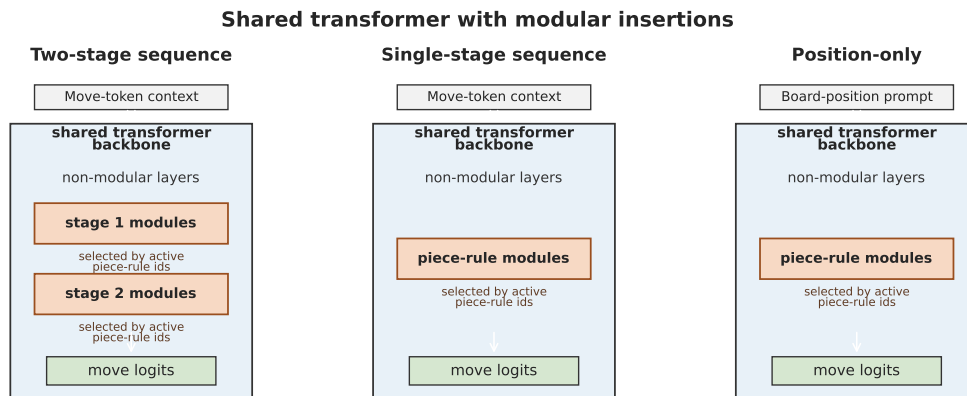


Figure 2: Model families evaluated. Each model is a shared transformer backbone with modular insertions at fixed points in the stack; the orange blocks are selected module capacity, not separate independent models. The sequence models consume move histories, while the position-only model consumes a board-position prompt.

Two-stage sequence model. The first architecture processes move-sequence context and inserts two sets of piece-rule module slots. The first stage is intended to propagate rule-relevant information into the representation and the second stage is closer to the output and is intended to influence move decisions. This design tests whether a deeper modular pathway improves rule-localized behavior.

Single-stage sequence model. The second architecture removes the second modular stage and keeps one set of piece-rule module slots. This reduces the number of modular mechanisms and tests whether a simpler design is easier to train and interpret.

Single-stage position model. The third architecture receives a board-position prompt rather than a move-sequence prefix. This tests whether sequence history lets the model infer rule context in a way that weakens dependence on modules. The position-only model is still trained to predict the next move, but its input is the current board state plus selected module slots rather than the full move history.

In all modular models, the selected module slots are determined by the active piece-rule identifiers. This is not learned dynamic routing; the active rule vector determines which modular capacity is loaded or selected.

4 Training

The training objective is next-token cross-entropy over legal game continuations. Standard chess remains a substantial part of the distribution so that the model retains ordinary chess competence, while variant data provides rule-specific pressure. The high-level curriculum is shown in Figure 1 and we also use pressures intended to discourage the shared layers from absorbing all rule-specific behavior in using lower effective learning rates for shared layers, module-specific learning-rate multipliers, and later training phases in which module-only and shared-only updates are separated.

After the main rule-family training, we train held-out K5 and N5 module slots while freezing shared layers and non-held-out modules. The held-out training examples come from games where the corresponding held-out rule is active and then running a short polish pass with the full model loaded but with shared layers and other modules still frozen. This pass is intended to reduce interference on other moves that occur in games where held-out modules are active, not merely to improve the held-out piece moves themselves.

The final held-out evaluations use matched samples of 3,840 examples across standard, seen-variant, held-out active-piece, held-out unique-geometry, and held-out context subsets.

Table 1: Polished held-out K5+N5 modules compared with base normal modules on the matched full evaluation. Values are base $\rightarrow$ polished.

Model	Overall top-4 $\uparrow$	Unique geometry loss $\downarrow$	Context loss $\downarrow$
Single-stage sequence	0.918 $\rightarrow$ 0.932	1.660 $\rightarrow$ 0.798	0.855 $\rightarrow$ 0.767
Two-stage sequence	0.940 $\rightarrow$ 0.952	1.816 $\rightarrow$ 0.764	0.708 $\rightarrow$ 0.700
Single-stage position	0.801 $\rightarrow$ 0.838	3.009 $\rightarrow$ 0.846	1.431 $\rightarrow$ 0.952

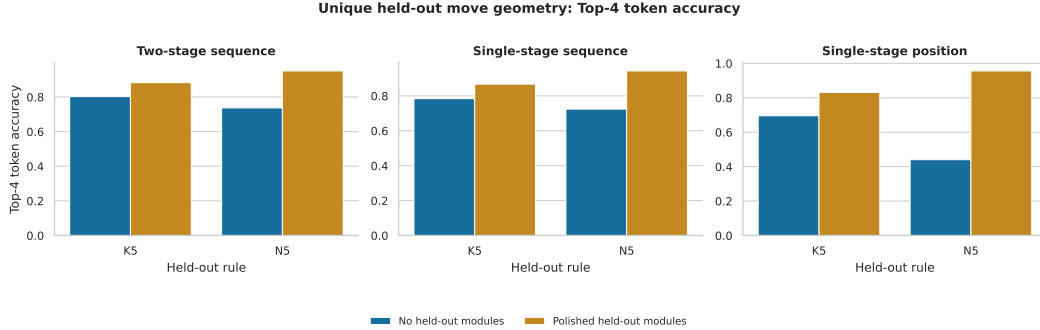


Figure 3: Top-4 token accuracy on K5 and N5 unique move geometries. Polished held-out modules improve unique-rule behavior relative to the base model without held-out modules.

5 Evaluation

We use four complementary evaluations.

Rule-conditional next-move performance. We measure exact move accuracy, token accuracy, top-4 token accuracy, loss, and component accuracies. Top-4 token accuracy is useful as move tokens can be decomposed into piece and geometry components, and exact-move accuracy is strict.

Unique held-out geometry. We isolate K5 and N5 moves whose source-to-target geometry is absent from seen rules as this should directly test whether the trained held-out module contributes behavior that the base model could not easily express using seen rule modules.

Context interference. Held-out rules are active for an entire game, but not every move in such a game is made by the held-out piece. We therefore evaluate moves by other pieces in held-out-rule games so that if loading a held-out module damages these moves, the adaptation is not clean even if rule-specific moves improve.

Wrong-module stress tests. We evaluate the same held-out unique moves while replacing the moving piece’s held-out module selection with the standard module or seen variant modules testing whether performance depends on the correct held-out module slot rather than on information stored entirely in shared layers.

6 Results

6.1 Held-out modules learn rule-specific behavior

The held-out modules appears to improve performance on held-out rule settings. In the full polished evaluation, moving from the base normal modules to polished K5+N5 modules improves overall loss and top-4 accuracy across all three model families with the strongest gains appearing on held-out unique geometry and held-out active-piece moves (Table 1). For the position model, unique-geometry loss falls from 3.01 to 0.85 and top-4 accuracy rises from 0.57 to 0.89. The sequence models also improve, though exact accuracy remains more modest than the others.

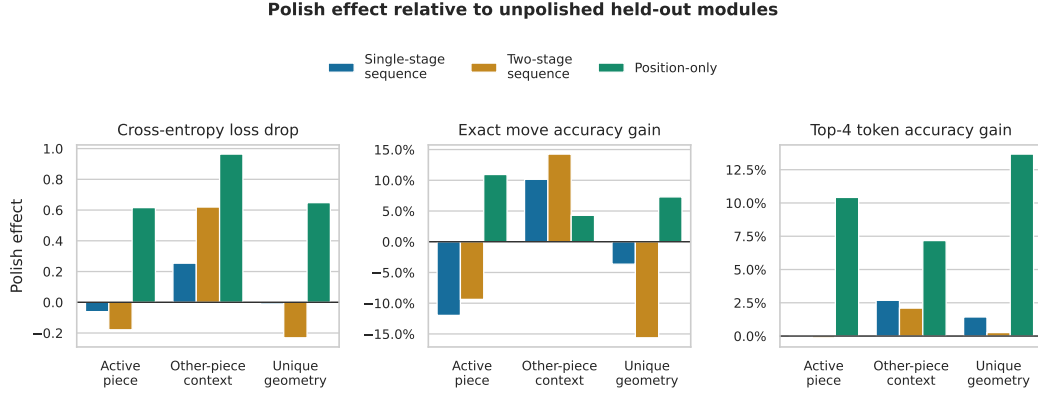


Figure 4: Effect of polishing relative to unpolished held-out modules. Each panel has its own axis: loss is plotted as a loss drop, while exact and top-4 accuracies are plotted as gains, so positive values always indicate improvement from polishing.

6.2 Polishing reduces context interference

Training held-out modules only on the held-out piece improves rule-specific moves but seems to interfere with other moves in games where the held-out rules are active, but the matched polished-vs-unpolished evaluation shows that a short polish pass helps to reduce this problem. For the held-out other-piece context subset, a short polish pass reduces loss from 1.02 to 0.77 for the single-stage sequence model, from 1.32 to 0.70 for the two-stage sequence model, and from 1.92 to 0.95 for the position model. The same comparison is summarized in Figure 4, where loss, exact accuracy, and top-4 accuracy are separated so that the direction and units of each metric are explicit.

6.3 Wrong modules destroy held-out geometry

The wrong-module stress test seems to give the clearest evidence that the held-out modules contain functionally relevant information. On 1,024 held-out unique moves, correct held-out modules produce nonzero exact accuracy and nonzero from-to geometry accuracy and replacing the moving piece’s module with the standard module or a seen variant module drives exact accuracy to zero for all three model families with from-to geometry accuracy additionally dropping to zero (Figure 5). This result would seem to be difficult to explain if the behavior were stored entirely in shared layers.

6.4 Dense fine-tuning baselines for new rules

To test whether the modular adaptation results are merely a consequence of exposing any transformer to held-out rule data, we also trained a dense transformer without modular rule slots or modularization pressure. The dense model was first trained on standard chess and then adapted to held-out rule data using several fine-tuning protocols: full-weight target-only updates, LoRA target-only updates, wider mixed-data updates, and longer full-data variants. We then evaluated sequential adaptation in which the model was first adapted to N5 and then adapted again to K5.

The dense baseline exhibits a sharp tradeoff between learning the new rule geometry and preserving existing behavior on ordinary, non-target moves. Target-only dense fine-tunes learn some held-out unique moves, but strongly damage this non-target context behavior. For example, the LoRA target-full K5-after-N5 run reaches 21.1% exact accuracy on held-out unique geometry, but non-target context exact accuracy falls from the dense base value of 12.3% to 1.2%. Full target-sparse fine-tuning has the same pattern, with non-target context exact accuracy falling to 1.6%. Conversely, wider dense fine-tunes preserve non-target behavior better but largely fail to learn the unique geometry: the full wide-full K5-after-N5 run keeps context exact accuracy at 19.1%, but held-out unique exact accuracy is only 1.2%. LoRA does not remove this tradeoff; it changes the balance slightly, but remains an entangled dense update.

This distinction is important as the modular comparison should be read as an adaptation-damage comparison on non-target moves. In the two-stage sequence model, adapting held-out modules raises

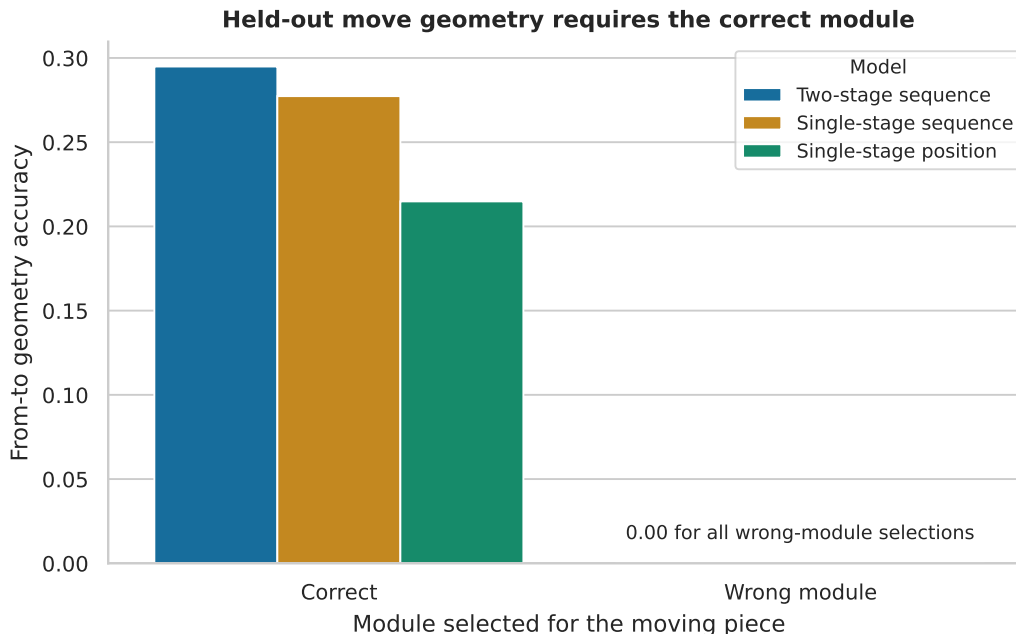


Figure 5: Wrong-module stress test. Selecting the correct held-out module preserves held-out from-to geometry accuracy on K5/N5 unique moves, while selecting a standard or seen-rule module eliminates it.

held-out unique exact accuracy from 1.6% to 37.1%, while non-target context exact accuracy changes from 26.4% to 18.2%. Dense target-only fine-tuning also improves held-out unique moves, but the updates that learn the new rule produce substantially larger damage to non-target moves, showing a less favorable adaptation tradeoff.

Table 2: Dense fine-tuning compared with modular adaptation on K5-after-N5 evaluation. The non-target columns measure ordinary moves by non-held-out pieces in held-out-rule contexts. Target-only dense updates learn some new geometry but damage non-target behavior, while wide dense updates preserve non-target behavior better but fail on held-out unique geometry.

Model	Non-target exact	Non-target top-4	Unique exact	Unique top-4
Dense base	12.3	90.0	0.0	72.6
Dense full target sparse	1.6	87.3	18.4	89.1
Dense LoRA target sparse	1.8	89.0	15.2	91.5
Dense LoRA target full	1.2	86.3	21.1	94.8
Dense full wide full	19.1	92.8	1.2	77.5
Dense LoRA wide sparse	13.3	90.4	0.0	73.4
Modular two-stage sequence, standard	26.4	96.2	1.6	75.6
Modular single-stage sequence, standard	19.3	94.3	0.4	75.5
Modular position, standard	10.0	86.2	0.4	56.9
Modular two-stage sequence, adapted	18.2	95.9	37.1	96.5
Modular single-stage sequence, adapted	14.6	93.1	27.0	93.5
Modular position, adapted	7.8	85.3	10.5	74.0

Reading Table 2 in terms of deltas makes the tradeoff more explicit. Dense target-only fine-tunes gain held-out geometry but lose ordinary behavior: the dense LoRA target-full run gains 21.1 percentage points of unique exact accuracy and 22.2 points of unique top-4 accuracy, but loses 11.1 points of non-target exact accuracy and 3.7 points of non-target top-4 accuracy. Dense wide fine-tuning moves in the opposite direction: the full wide-full run gains 6.8 points of non-target exact accuracy and 2.8 points of non-target top-4 accuracy, but gains only 1.2 points of unique exact accuracy. The modular

models occupy a different region of this tradeoff. The two-stage sequence model gains 35.5 points of unique exact accuracy and 20.9 points of unique top-4 accuracy when the adapted modules are loaded, while losing 8.2 points of non-target exact accuracy and only 0.3 points of non-target top-4 accuracy. The single-stage sequence model has a smaller non-target exact loss of 4.7 points while still gaining 26.6 points of unique exact accuracy. Thus the modular result is not absence of side effects, but a better adaptation tradeoff: more held-out rule learning per unit of non-target damage, with the adapted behavior localized in loadable components.

Figure 6 visualizes these deltas directly. The x-axis measures damage or improvement on ordinary non-target moves in held-out-rule contexts, while the y-axis measures gain on held-out unique geometry. This makes the position-only result easier to interpret: it is weaker in absolute accuracy, but its adapted module still moves upward with relatively little movement leftward. The central interpretation is therefore an adaptation tradeoff rather than a pure accuracy ranking: the modular models do not just learn new rules, they learn them in components that can be loaded without rewriting the whole policy.

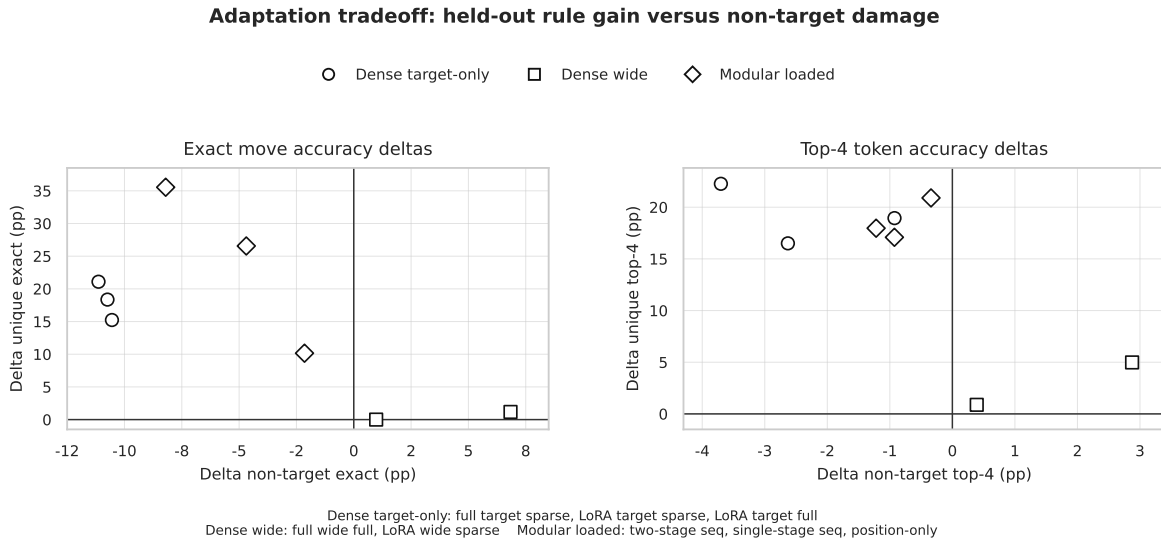


Figure 6: Delta tradeoff for adapting to K5 after N5. Each point is measured relative to the relevant base: the dense base checkpoint for dense fine-tunes, and the same modular model with standard modules loaded for modular adaptation. Upward movement means better held-out unique-rule performance; leftward movement means more damage on ordinary non-target moves in held-out-rule contexts. The position-only model is weaker in absolute terms, but its delta shows meaningful held-out-rule learning with comparatively small non-target damage.

6.5 Training dynamics

Training curves show that the models learn the next-move task and that top-4 token accuracy becomes reasonably high even when exact move accuracy remains lower. This should probably be expected as a move is represented by several tokens in sequence. Figure 7 shows representative training curves from the run histories.

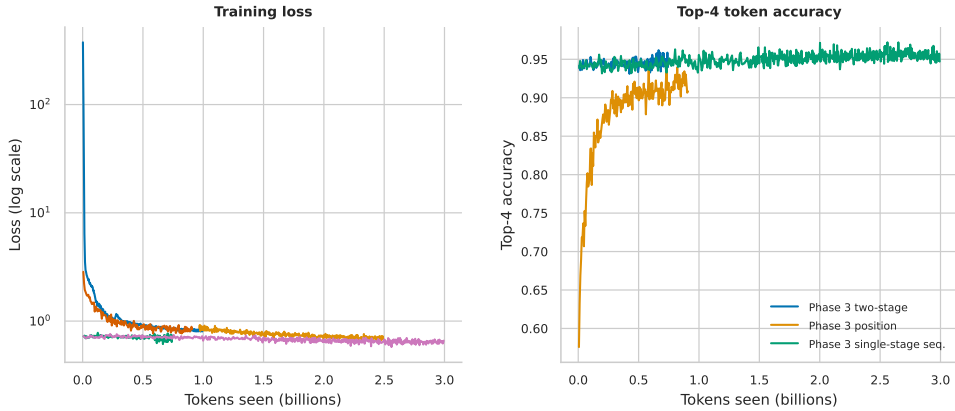


Figure 7: Training dynamics for major runs. Later phases and model variants are not directly controlled comparisons, but the curves document scale, convergence, and throughput.

7 Discussion

The results support a limited but meaningful form of functional modularity. The goal was not to obtain clean symbolic isolation where each rule is represented as an independently interpretable program, but instead the goal was to make targeted in-place changes to model behavior have lower side effects than ordinary dense updates. On this criterion the modular models are reasonably successful in part: wrong-module selections destroy held-out move geometry, correct held-out modules improve unique-rule performance, and standard-rule behavior is not rewritten when held-out modules are inactive. At the same time, the system is not a clean plug-and-play symbolic rule engine as unpolished held-out modules interfere with unrelated moves in held-out-rule games, indicating that rule-specific adaptation still interacts with shared context representations and output decoding. The polish pass is important as it shows this interference can be reduced with a reasonably small number of overall tokens while keeping the rest of the model frozen.

The comparison among architectures is additionally informative as the position-only model is worse in absolute next-move accuracy, but polishing improves its held-out unique and context performance strongly. This is consistent with the idea that removing move history reduces one path by which the model can infer rule context from sequence patterns alone, meaning that the sequence models are stronger overall, but they can potentially rely more on contextual regularities outside the modules. The two-stage model has the best ordinary performance among the evaluated models, but its held-out exact accuracy is more sensitive to polishing tradeoffs.

The dense baselines help to elucidate the advantages the modular models provide for learning new rules. Dense target-only fine-tuning can learn held-out rule geometry, but the update is entangled with the rest of the policy as the strongest dense target runs improve unique held-out moves while substantially damaging ordinary non-target moves in held-out-rule contexts. Dense wide fine-tuning has the opposite behavior, preserving ordinary moves better while learning little of the unique held-out geometry. The modular models do not eliminate this tradeoff, but they would seem to move it to a better region with the sequence modular models gaining more held-out unique accuracy per unit of non-target damage, and the change being localized in components that can be loaded or left inactive. The position-only modular model is weaker in absolute accuracy, but it still shows the same low-interference pattern relative to its own baseline.

This is the main sense in which the result should be read as modular rather than merely accurate. The modular models are more complex to train and evaluate than the dense model, and they rely

on externally selecting the appropriate module rather than discovering the ruleset from input alone, however, they provide a mechanism for targeted adaptation that dense fine-tuning lacks as new behavior can be introduced through a small loadable component without rewriting all shared weights. These results suggest that structured rule-family training can induce some degree of functional modularity, but the pressure as applied here is incomplete. In domains such as programming languages, where local syntax and semantics vary systematically across related rule systems, a similar approach might help reduce interference during adaptation, like if one wanted to update a model to perform better with a newer version of a programming language or wanted to fine tune on a specific programming style used within an organization. Chess variants are far more controlled than language tasks using natural software ecosystems, therefore the current evidence should be interpreted as a proof of concept rather than a general solution.

8 Limitations

This work has several limitations. First, the environment is synthetic and highly structured, chess rule variants are useful for measurement, but they do not capture the ambiguity, distribution shift, or scale of language and code tasks. Second, we report a limited number of training runs rather than statistically powered repetitions across seeds. Third, our module selection is provided by the known ruleset; the model is not required to infer which rule system is active from raw observations alone. Fourth, the held-out adaptation protocol demonstrates partial localization but not perfect noninterference: context interference is measurable and must be reduced by polishing. Finally, the generated data and training pipeline are substantial, and reproducing the full training process requires significant GPU time.

9 Conclusion

Structured rule variation provides a practical way to test whether transformers can be pressured toward modular behavior at training time without forced routing. In controlled chess-rule families, piece-rule modules acquire functionally relevant behavior with held-out modules improving held-out move prediction, and wrong module selections nearly eliminating unique held-out move geometry. The strongest caution is that modularity is partial, new modules can interfere with unchanged play unless they are polished in context. Overall, these experiments suggest that rule-family training can create measurable modular structure in transformers, while also showing that architectural modularity alone is not enough to guarantee clean isolation.